

Quantum Multi-Agent Reinforcement Learning via Variational Quantum Circuit Design

[†]Won Joon Yun, [†]Yunseok Kwak, [†]Jae Pyoung Kim, [§]Hyunhee Cho,

[‡]Soyi Jung, [°]Jihong Park, and [†]Joongheon Kim

[†]School of Electrical Engineering, Korea University, Seoul, Republic of Korea

[§]School of Electronic and Electrical Engineering, Sungkyunkwan University, Suwon, Republic of Korea

[‡]School of Software, Hallym University, Chuncheon, Republic of Korea

[°]School of Information Technology, Deakin University, Geelong, Victoria, Australia

Abstract—In recent years, quantum computing (QC) has been getting a lot of attention from industry and academia. Especially, among various QC research topics, variational quantum circuit (VQC) enables quantum deep reinforcement learning (QRL). Many studies of QRL have shown that the QRL is superior to the classical reinforcement learning (RL) methods under the constraints of the number of training parameters. This paper extends and demonstrates the QRL to quantum multi-agent RL (QMARL). However, the extension of QRL to QMARL is not straightforward due to the challenge of the noise intermediate-scale quantum (NISQ) and the non-stationary properties in classical multi-agent RL (MARL). Therefore, this paper proposes the centralized training and decentralized execution (CTDE) QMARL framework by designing novel VQCs for the framework to cope with these issues. To corroborate the QMARL framework, this paper conducts the QMARL demonstration in a single-hop environment where edge agents offload packets to clouds. The extensive demonstration shows that the proposed QMARL framework enhances 57.7% of total reward than classical frameworks.

Index Terms—Quantum deep learning, Multi-agent reinforcement learning, Quantum computing

I. INTRODUCTION

The recent advances in computing hardware and deep learning algorithms have spurred the ground-breaking developments in distributed learning and multi-agent reinforcement learning (MARL) [1]. The forthcoming innovations in quantum computing hardware and algorithms will accelerate or even revolutionize this trend [2], motivating this research on quantum MARL (QMARL). Indeed, quantum algorithms have huge potential in reducing model parameters without compromising accuracy by taking advantage of quantum entanglement [3]. A remarkable example is the variational quantum circuit (VQC) architecture, also known as a quantum neural network (QNN) [4], [5], which integrates a quantum circuit into a classical deep neural network. The resultant hybrid quantum-classical model enables quantum reinforcement learning (QRL) that is on par with classical reinforcement learning with more model parameters [6], [7], which can accelerate the training and inference speed while saving computing resources [8]. Inspired from this success, in this article we aim to extend QRL to QMARL by integrating VQC into classical MARL. This problem is non-trivial due to the

trade-off between quantum errors and MARL training stability as we shall elaborate next.

In MARL, each agent interacts with other agents in a cooperative or competitive scenario. Such agent interactions often incur the non-stationary reward of each agent, hindering the MARL training convergence. A standard way to cope with this MARL non-stationarity is the centralized training and decentralized execution (CTDE) method wherein the reward is given simultaneously to all agents by concatenating their state-action pairs. In this respect, one can naively implement a VQC version of CTDE as in [6]. Unfortunately, since QRL under VQC represents the state-action pairs using qubits, such a naïve CTDE QMARL implementation requires the qubits increasing with the number of agents, and incurs the quantum errors increasing with the number of qubits [9], hindering the MARL convergence and scalability. Under the current noise intermediate-scale quantum (NISQ) era (up to a few hundreds qubits), it is difficult to correct such type of quantum errors due to the insufficient number of qubits. Instead, the quantum errors brought on by quantum gate operations can be properly controlled under NISQ [9]. Motivated from this, we apply a quantum state encoding method to CTDE QMARL, which reduces the dimension of the state-action pairs by making them pass through a set of quantum gates.

Contributions. The major contributions of this research can be summarized as follows.

- We propose a novel QMARL by integrating CTDE and quantum state encoding into VQC based MARL.
- By experiments, we demonstrate that the proposed QMARL framework achieves 57.7% higher total rewards compared to classical MARL baselines under a multiple edge-to-cloud queue management scenario.

II. QUANTUM COMPUTING AND CIRCUIT

A. Quantum Computing in a Nutshell

Quantum computing utilizes a *qubit* as the basic unit of computation. The qubit represents a quantum superposition state between two basis states, which denoted as $|0\rangle$ and $|1\rangle$. Mathematically, there are two ways to describe a qubit state:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \text{ where } \|\alpha\|_2^2 + \|\beta\|_2^2 = 1$$

$$|\psi\rangle = \cos(\delta/2)|0\rangle + e^{i\varphi} \sin(\delta/2)|1\rangle, \forall \delta, \varphi \in [-\pi, \pi].$$

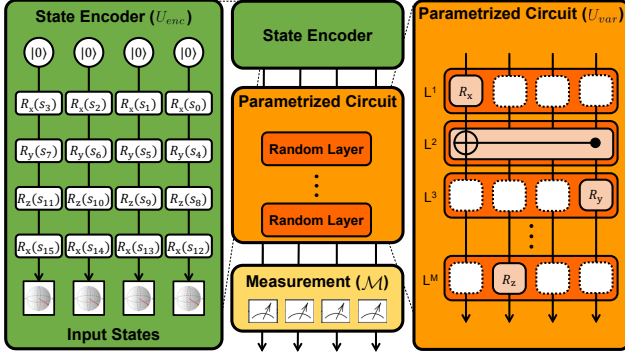


Fig. 1: The illustration of VQC.

The former is based on a normalized 2D complex vector, while the latter is based on polar coordinates (δ, φ) from a geometric viewpoint. The qubit state is mapped into the surface of a 3-dimensional unit sphere, which is called *Bloch sphere*. In addition, a quantum gate is a unitary operator transforming a qubit state. For example, $R_x(\delta)$, $R_y(\delta)$, and $R_z(\delta)$ are rotation operator gates by rotating δ around their corresponding axes in the Bloch sphere. These gates are dealing with a single qubit. In contrast, there are quantum gates which operate on multiple qubits, called controlled rotation gates. They act on a qubit according to the signal of several control qubits, which generates quantum entanglement between those qubits. Among them, a *Controlled-X* gate is one of widely used control gates which changes the sign of the second qubit if the first qubit is $|1\rangle$. These gates allow quantum algorithms to work with their features on VQC, which will be for QMARL.

B. Variational Quantum Circuit (VQC)

VQC is a quantum circuit that utilizes learnable parameters to perform various numerical tasks, including estimation, optimization, approximation, and classification. As shown in Fig. 1, the operation of the general VQC model can be divided into three steps. The first one is *state encoding step* U_{enc} , and in this step, a classical input information is encoded into corresponding qubit states, which can be treated in the quantum circuit. The next step is *variational step* U_{var} , and it is for entangling qubit states by controlled gates and rotating qubits by parameterized rotation gates. This process can be repeated in multi-layers with more parameters, which enhances the performance of the circuit. The last one is *measurement step* $\mathcal{M}$, which measures the expectation value of qubit state according to its corresponding computational bases. This process can be formulated as follows:

$$f(x; \theta) = \otimes \Pi_{M \in \mathcal{M}} \langle 0 | U_{enc}^\dagger(x) U_{var}^\dagger(\theta) M U_{var}(\theta) U_{enc}(x) | 0 \rangle,$$

where $\otimes$ stands for the qubit superposition operator; $f(x; \theta)$ is the output of VQC with inputs x and circuit parameter θ ; $\mathcal{M}$ is the set of quantum measurement bases in VQC with $|\mathcal{M}| \leq n_{qubit}$ where n_{qubit} is the number of qubits. The example of the state encoder in Fig. 1 can be expressed as follows:

$$U_{enc}(s_0, s_4, s_8, s_{12}) = R_x(s_{12}) \cdot R_z(s_8) \cdot R_y(s_4) \cdot R_x(s_0).$$

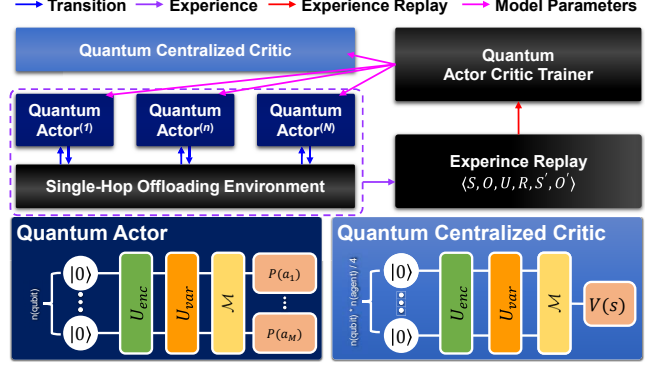


Fig. 2: The structure of QMARL framework.

The quantum circuit parameters are updated every training epoch, toward the direction of optimizing the objective output value from VQC. Through this process, VQC is known to be able to approximate any continuous function, which is similar to classical neural network computation [10]. Therefore, VQC is also called *quantum neural network (QNN)* [11]. In this paper, two different VQCs are used to approximate the optimal actions of actor and the accurate state value of critic.

III. QUANTUM MARL FRAMEWORK

A. QMARL Architecture

Our proposed QMARL is decentralized for scalability, every agent in the QMARL has a VQC-based policy, i.e., agents do not require communications among agents. Fig. 2 shows the VQC that is used in quantum actor (refer to Sec. III-A1) and critic (refer to Sec. III-A2).

1) *Quantum Actor*: For the quantum actor, the VQC will be used to calculate the probabilities of actions of each agent. Then, the quantum policy is written as follows:

$$\pi_\theta(u_t | o_t) = \text{softmax}(f(o_t; \theta)),$$

where $\text{softmax}(\mathbf{x}) \triangleq \left[\frac{e^{x_1}}{\sum_{i=1}^N e^{x_i}}, \dots, \frac{e^{x_i}}{\sum_{i=1}^N e^{x_i}}, \dots, \frac{e^{x_N}}{\sum_{i=1}^N e^{x_i}} \right]$. At time t , the actor policy of the n -th agent makes action decision with the given observation o_t^n , which is denoted as $\pi_{\theta^n}(a_t^n | o_t^n)$. Note that θ^n denotes parameters of n -th actor. Then, the action u_t^n is computed as follows:

$$u_t^n = \arg \max_u \pi_{\theta^n}(u_t^n | o_t^n).$$

2) *Quantum Centralized Critic*: We adopt the centralized critic for CTDE as a state-value function. At time t , the parameterized critic estimates the discounted returns given s_t as follows:

$$V^\psi(s_t) = f(s_t; \psi) \simeq \mathbb{E} \left[\sum_{t'=t}^T \gamma^{t'-t} \cdot r(s_{t'}, \mathbf{u}_{t'}) \mid s_t = s \right],$$

where γ , T , $\mathbf{u}_t$, and $r(s_{t'}, \mathbf{u}_{t'})$ stand for a discounted factor $\gamma \in [0, 1)$, an episode length, the actions of all agents, and reward functions, respectively. In addition, ψ presents trainable parameters of a critic. Note that s_t is the ground truth at t . Note that the state encoding is used as shown in green box in Fig. 1 because the state size is larger than the size in observation.

Algorithm 1: CTDE-based QMARL Training

```

1 Initialize the parameters of actor-critic networks and the replay
  buffer;  $\Theta \triangleq \{\theta^1, \dots, \theta^N\}$ ,  $\psi, \phi, \mathcal{D} = \{\}$ ;
2 repeat
3    $t = 0, s_0 = \text{initial state}$ ;
4   while  $s_t \neq \text{terminal}$  and  $t < \text{episode limit}$  do
5     for each agent  $n$  do
6       Calculate  $\pi_{\theta^n}(u_t^n | o_t^n)$  and sample  $u_t^n$ ;
7     end
8     Get reward  $r_t$  and next state and observations  $s_{t+1}$ ,
        $\mathbf{o}_{t+1} = \{o_t^1, \dots, o_t^N\}$ ;
        $\mathcal{D} = \mathcal{D} \cup \{(s_t, \mathbf{o}_t, \mathbf{u}_t, r_t, s_{t+1}, \mathbf{o}_{t+1})\}$ ;
        $t = t + 1$ , step = step + 1;
9   end
10  for each timestep  $t$  in each episode in batch  $\mathcal{D}$  do
11    Get  $V^\psi(s_t)$ ;  $V^\phi(s_{t+1})$ ;
12    Calculate the target  $y_t$ ;
13  end
14  Calculate  $\nabla_{\Theta} J$ ,  $\nabla_{\psi}$ , and update  $\Theta, \psi$ ;
15  if target update period then
16    Update the target network,  $\phi \leftarrow \psi$ 
17  end
18 until obtaining optimal policies;

```

TABLE I: The MDP of a single-hop offloading environment.

Observation	$o_t^n \triangleq \{q_t^{e,n}, q_{t-1}^{e,n}\} \cup_{k=1}^K \{q_t^{c,k}\}$
Action	$u_t^n \in \mathcal{A} \triangleq \mathcal{I} \times \mathcal{P}$
o Destination space	$\mathcal{I} \triangleq \{1, \dots, K\}$
o Packet amount space	$\mathcal{P} \triangleq \{p_{\min}, \dots, p_{\max}\}$
State	$s_t \triangleq \cup_{n=1}^N \{o_t^n\}$
Reward	$r(s_t, \mathbf{u}_t)$ in (1)

TABLE II: The experiment parameters.

Parameters	Values
The numbers of clouds and edge agents (K, N)	2, 4
The packet amount space ($\mathcal{P}$)	$\{0.1, 0.2\}$
The hyper-parameters of environment ($w_{\mathcal{P}}, w_{\mathcal{R}}$)	(0.3, 4)
Transmitted packets from the cloud ($b_t^{c,k}$)	0.3
The capacity of queue ($q_{\max}$)	1
Optimizer	Adam
The number of gates in U_{var}	50
The number of qubits of actor/critic	4
Learning rate of actor/critic	$1 \times 10^{-4}, 1 \times 10^{-5}$

IV. EXPERIMENTS AND DEMONSTRATIONS

A. Single-Hop Offloading Environment

The environment used in this paper consists of K clouds and N edges. The clouds and edges have queues q^c and q^e that temporally store packets. All edge agents offload their packets to clouds. The queue dynamics are as follows:

$$q_{t+1}^{i,k} = \text{clip}(q_t^{i,k} - u_t^{i,k} + b_t^{i,k}, 0, q_{\max}),$$

where the superscript $i \in \{c, e\}$ identifies the cloud and an edge device. The terms $u_t^{i,k}$ and $b_t^{i,k}$ imply the total transmitting packet size and the packet arrival of k -th cloud or n -th edge, respectively. Note that $u_t^{e,n}$ is n -th edge agent's action. In addition, a clipping function is defined as $\text{clip}(x, x_{\min}, x_{\max}) \triangleq \min(x_{\max}, \max(x, x_{\min}))$.

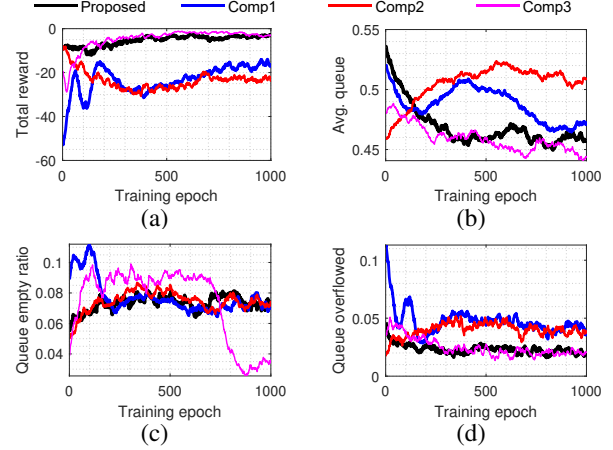


Fig. 3: The experimental result of various metrics with comparing different MARL frameworks.

B. Training

The objective of MARL agents is to maximize the discounted returns. To derive the gradients, we leverage the joint state-value function V^ψ . Our framework uses an multi-agent policy gradient (MAPG), which can be formulated as follows:

$$\nabla_{\theta^n} J = -\mathbb{E}_{\pi} \left[\sum_{t=1}^T \sum_{n=1}^N y_t \nabla_{\theta^n} \log \pi_{\theta}(u_t^n | o_t^n) \right], \quad \nabla_{\psi} J = \nabla_{\psi} \sum_{t=1}^T \|y_t\|^2$$

s.t. $y_t = r(s_t, \mathbf{u}_t) + \gamma V^\phi(s_{t+1}) - V^\psi(s_t)$, and ϕ is the parameters of target critic. The detailed procedure is in **Algorithm 1**.

In this paper, we assume that the capacities of edges and clouds are all limited to $q_{\max}$ and edge agents receive packets from previous hops, where the distribution is uniform $\forall b_t^{e,n} \sim \mathcal{U}(0, w_{\mathcal{P}} \cdot q_{\max})$. The objective of this scenario is to minimize the total amount of overflowed queue and the event that the queue is empty. Thus, the reward $r(s_t, \mathbf{u}_t)$ can be as follows:

$$- \sum_{k=1}^K \left[\mathbb{I}_{(q_{t+1}^{c,k}=0)} \cdot \tilde{q}_t^{c,k} + \mathbb{I}_{(q_{t+1}^{c,k}=q_{\max})} \cdot \hat{q}_t^{c,k} \cdot w_{\mathcal{R}} \right], \quad (1)$$

s.t. $\tilde{q}_t^{c,k} = |q_t^{c,k} - u_t^{c,k} + b_t^{c,k}|$ and $\hat{q}_t^{c,k} = |q_{\max} - \tilde{q}_t^{c,k}|$, where $w_{\mathcal{R}}$ is the hyperparameter of rewards. Note that $r(s_t, \mathbf{u}_t) \in [-\infty, 0]$ (negative) because we consider the occurrence of abnormal queue states (e.g., queue overflow or underflow) as a negative reward. The Markov decision process (MDP) of this environment is presented in Table I.

C. Experimental and Demonstration Setup

To verify the effectiveness of the proposed QMARL framework (named, **Proposed**), we compare our proposed QMARL with two comparing methods. Here, ‘Comp1’ is a CTDE hybrid QMARL framework where the actors use a VQC-based policy and the centralized critic uses a classical neural networks. In addition, ‘Comp2’ is a CTDE classical MARL framework that is not related to quantum algorithms. Note that the trainable parameters of these three frameworks are all set to 50 for actor and critic computation. Lastly, ‘Comp3’ is a

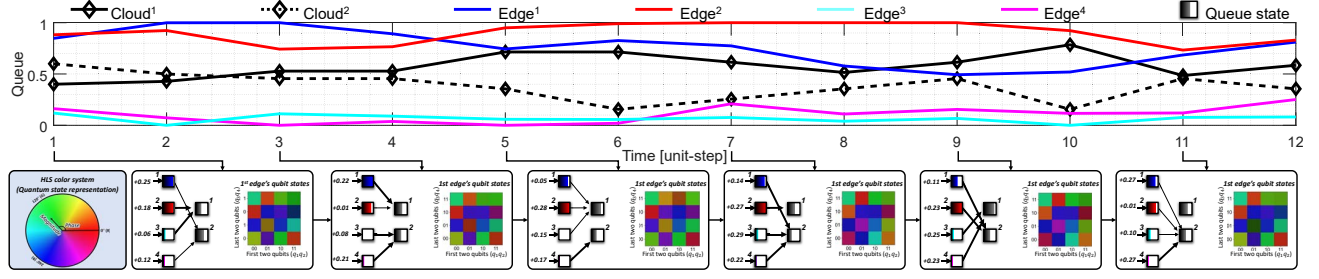


Fig. 4: The demonstration of QMARL framework.

classical CTDE MARL where the number of parameters is more than 40K. The simulation parameter settings are listed in Table II. We use python libraries (e.g., `torchquantum` and `pytorch`) for deploying VQCs and DL methods, which support GPU acceleration [12]. In addition, all experiments are conducted on AMD Ryzen™ Threadripper™ 1950x and NVIDIA RTX 3090. We have confirmed that the training time of QMARL for 1,000 epochs is not expensive (≈ 35 minutes).

D. Evaluation Results

1) *Reward Convergence*: Fig. 3 presents the demonstration results. As shown in Fig. 3(a), the reward of QMARL frameworks is around -3.0 for Proposed and -16.6 for Comp1, whereas the classical MARL frameworks record -22.5 for Comp2 and -2.8 for Comp3, respectively. We calculate the achievability as min-max normalization with the average returns of random walk. Note that the random walk records -33.2 on average. The achievability of QMARL frameworks is 90.9% for Proposed and 49.8% for Comp1. However, the classical MARL frameworks achieve 33.2% for Comp2 and 91.5% for Comp3. In summary, the proposed QMARL outperforms hybrid QMARL and classical MARL under the constraint of the number of trainable parameters.

2) *Performance*: The average queue states of edges/clouds and clouds are 0.460 for Proposed, 0.480 for Comp1, 0.510 for Comp2, and 0.453 for Comp3, respectively. The ratio of the number of empty queue events records in a high order of Comp2, Comp1, Proposed, and Comp3. However, the overflowed queue is low with the order of Proposed, Comp3, Comp2, and Comp1. According to Fig. 3(a-d), the QMARL framework outperforms both classical and hybrid quantum-classical MARL frameworks under the constraints of the number of trainable parameters.

E. Demonstration

Due to high network latency of utilizing quantum clouds, we conduct demonstration on simulation. Fig. 4 shows the visualization of the workflow of our QMARL framework. The superpositioned qubit states (i.e., magnitude and, phase vector) are expressed as 4×4 heatmap in hue-lightness-saturation color system. We provide source codes¹ including QMARL, the single-hop environment, and the simulator.

¹<https://github.com/WonJoon-Yun/Quantum-Multi-Agent-Reinforcement-Learning>

V. CONCLUDING REMARKS AND FUTURE WORK

This paper introduces quantum computing concepts to MARL, i.e., QMARL. To resolve the challenge of QMARL, we adopt VQC with state encoding and the concept of CTDE. From the single-hop environment, we verify the superiority of QMARL corresponding to the number of trainable parameters and the feasibility of QMARL. As a future work direction, the implementation of QMARL to the quantum cloud (e.g., IBM quantum, Xanadu, or IonQ) should be of interest because the impact of noise is considerable on quantum computing.

Acknowledgement. This research was supported by the National Research Foundation of Korea (2022R1A2C2C004869 and 2021R1A4A1030775). W. J. Yun and Y. Kwak contributed equally to this work. S. Jung, J. Park, and J. Kim are corresponding authors.

REFERENCES

- [1] J. Park, S. Samarakoon, A. Elgabri, J. Kim, M. Bennis, S.-L. Kim, and M. Debbah, "Communication-efficient and distributed learning over wireless networks: Principles and applications," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 796–819, 2021.
- [2] M. Schuld and N. Killoran, "Is quantum advantage the right goal for quantum machine learning?" *CoRR*, vol. abs/2203.01340, 2022.
- [3] S. Oh, J. Choi, and J. Kim, "A tutorial on quantum convolutional neural networks (QCNN)," in *Proc. of IEEE Int'l Conf. on ICT Convergence (ICTC)*, October 2020.
- [4] Z. Hong, J. Wang, X. Qu, X. Zhu, J. Liu, and J. Xiao, "Quantum convolutional neural network on protein distance prediction," in *Proc. IEEE Int'l Joint Conf. on Neural Networks (IJCNN)*, July 2021.
- [5] Y. Kwak, W. J. Yun, S. Jung, and J. Kim, "Quantum neural networks: Concepts, applications, and challenges," in *Proc. IEEE Int'l Conf. on Ubiquitous and Future Networks (ICUFN)*, August 2021.
- [6] S. Y.-C. Chen, C.-H. H. Yang, J. Qi, P.-Y. Chen, X. Ma, and H.-S. Goan, "Variational quantum circuits for deep reinforcement learning," *IEEE Access*, vol. 8, pp. 141 007–141 024, 2020.
- [7] Y. Kwak, W. J. Yun, S. Jung, J.-K. Kim, and J. Kim, "Introduction to quantum reinforcement learning: Theory and PennyLane-based implementation," in *Proc. IEEE Int'l Conf. on ICT Convergence (ICTC)*, October 2021.
- [8] G. Carleo, I. Cirac, K. Cranmer, L. Daudet, M. Schuld, N. Tishby, L. Vogt-Maranto, and L. Zdeborová, "Machine learning and the physical sciences," *Reviews of Modern Physics*, vol. 91, no. 4, p. 045002, 2019.
- [9] P. W. Shor, "Scheme for reducing decoherence in quantum computer memory," *Physical Review A*, vol. 52, no. 4, p. R2493, 1995.
- [10] J. Biamonte, "Universal variational quantum computation," *Physical Review A*, vol. 103, no. 3, p. L030401, 2021.
- [11] N. Wiebe, A. Kapoor, and K. M. Svore, "Quantum deep learning," *CoRR*, vol. abs/1412.3489, 2014.
- [12] H. Wang, Y. Ding, J. Gu, Z. Li, Y. Lin, D. Z. Pan, F. T. Chong, and S. Han, "QuantumNAS: Noise-adaptive search for robust quantum circuits," in *Proc. IEEE Int'l Symposium on High-Performance Computer Architecture (HPCA)*, April 2022.